Fundamentos de Javascript enfocado a React Native

DESARROLLO DE INTERFACES

Técnico de Grado Superior Desarrollo de Aplicaciones Multiplataforma



Pilar González Gómez
mgongom171@g.educaand.es

0. Software necesario

- Visual Studio Code
  https://code.visualstudio.com/

- Node.js
  https://nodejs.org/en/download

ÍNDICE

1. **Introducción a JavaScript**
2. **Características de JavaScript**
3. **Variables y datos**
4. **Operadores y control de flujo**
5. **Funciones**
6. **Ámbitos en JS**
7. **Arrays**
8. **Readline: entrada de datos**
9. **Sincronía**
10. **Promises**
11. **JSON: JavaScript Object Notation**
12. **Lectura de JSON local con Node.js**
13. **fetch() — Obtener datos de una API**
14. **Módulos: import / export**

1. Introducción a JavaScript

1.1. Qué es JavaScript

JavaScript (JS) es un lenguaje de programación utilizado originalmente para añadir comportamiento e interactividad a las páginas web.

Actualmente se utiliza en muchos ámbitos: Desarrollo web, aplicaciones móviles, servidores, aplicaciones de escritorio, automatización, videojuegos, desarrollo de interfaces.

Utilizaremos JavaScript como base para desarrollar posteriormente interfaces con React Native

1. Introducción a JavaScript

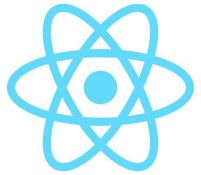
1.2. JavaScript, React y React Native



JAVASCRIPT

Lenguaje de programación

↓



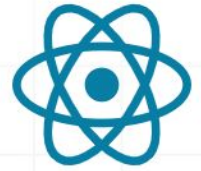
REACT

Librería para construir interfaces de usuario

↓

REACT NATIVE

Framework para desarrollar aplicaciones móviles multiplataforma



1. Introducción a JavaScript

1.3. Breve historia de JavaScript



1995: Brendan Eich (Netscape) crea JavaScript y lo llama ***Mocha***, luego ***LiveScript***. Renombrado a ***JavaScript*** (por razones de marketing).

1996: aparece ***JScript***, la implementación de Microsoft.

La existencia de **diferentes implementaciones** provoca problemas de compatibilidad.

JavaScript se estandariza mediante **ECMAScript.**

2005-2009: Ajax y jQuery popularizan JavaScript; surge ***Node.js***, expandiendo su uso al backend.

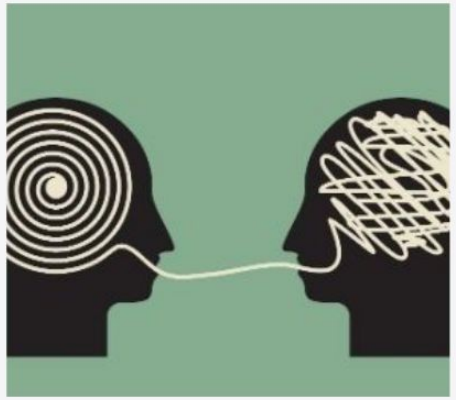
2015: ES6 (ECMAScript 2015) introduce mejoras significativas (arrow functions, clases, módulos).

Actualidad: ECMAScript continúa evolucionando.

2. Características de JavaScript

2.1. Características de JavaScript

Interpretado





Tipado dinámico

Rendimiento





Débilmente tipado

2. Características de JavaScript

2.2. Tipado dinámico ≠ tipado débil

TIPADO DINÁMICO ≠ DÉBILMENTE TIPADO

```
1   let x = 10;          // tipado dinámico
2   x = "10";            // puede cambiar de tipo
3
4   console.log(x + 5); // "105"
```

Tipado dinámico: el **tipo de una variable se determina durante la ejecución** y puede cambiar.
Débilmente tipado: permite **conversiones automáticas** entre tipos en determinadas operaciones.

2. Características de JavaScript

2.3. Motores JavaScript

JavaScript es **ejecutado** por un motor JavaScript que **interpreta y puede compilar y optimizar** el código durante la ejecución.

Los más conocidos son:

- V8 de Google usado en Chrome y Node.js
- Spider Monkey de Mozilla
- JavaScriptCore usado en Safari y productos de Apple
- Chakra de Microsoft

2. Características de JavaScript

2.4. Cómo se ejecuta JavaScript

Código JavaScript

↓

Tokens

↓

Parser (nodos)

↓

AST

↓

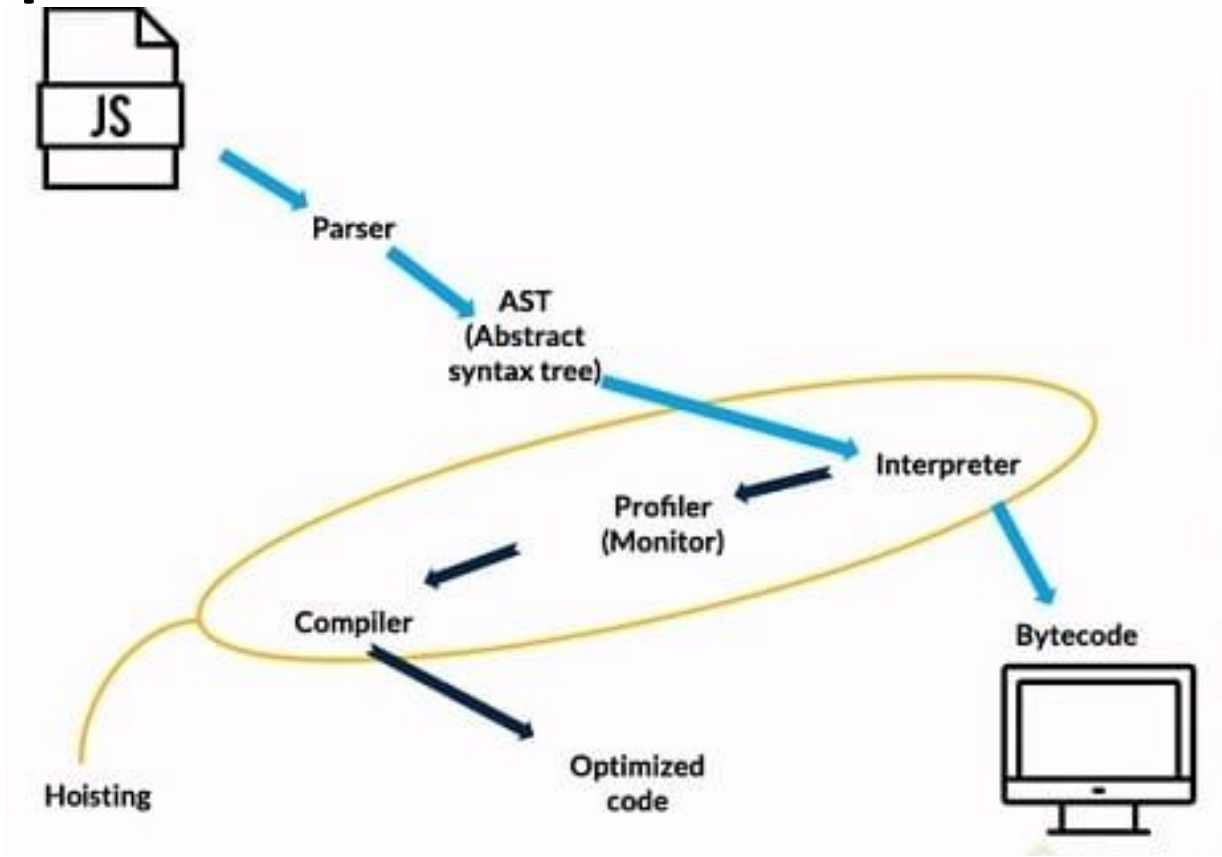
Interpretación / bytecode

↓

Optimización JIT

↓

Código máquina -> Ejecución



3. Variables y datos

3.1. Declaración de variables

var evitaremos su uso.

let ¡Sí! para variables cuyo valor puede cambiar

const para referencias que no se van a reasignar

```
let firstName="Pilar";
let lastName="Gonzalez";
const fullName=`${firstName} ${lastName}`;
console.log(fullName);
```

3. Variables y datos

3.2. Hoisting

Es el comportamiento por el que determinadas declaraciones son **procesadas** por JavaScript **antes de ejecutar el código** correspondiente.

Este efecto **se produce al declarar variables con var**, donde el intérprete reserva la memoria necesaria antes de ejecutar la primera línea del script. Solución**:** es **usar let y const** para declarar variables y evitar el uso de var.

También sucede con las **funciones**. Cuando declaramos funciones se reserva un espacio en memoria con el valor por defecto ***function***.

En JS **se puede llamar a una función antes de definirla y no se genera error de ejecución**. En este caso para evitar el hoisting tenemos que definir las funciones siempre antes de llamarlas.

Las **funciones anónimas** no producen hoisting.

Vídeo:
https://youtu.be/EvfRXyKa_GI?si=BuHp0erHE_ClsttQ

3. Variables y datos

3.3. Tipos de datos

Primitivos:

- **string**
- **number**
- **boolean**
- **undefined**
- **null**
- **bigint**
- **symbol**

https://es.javascript.info/types

No primitivos: entre otros **null** (es un objeto), **array**, **object, set, map**

3. Variables y datos

3.4. Typeof

Podemos utilizar typeof para conocer el tipo de un valor.

```
const edad = 25;

console.log(typeof edad);   //number
```

3. Variables y datos

3.5. Cadenas de texto

En JavaScript, hay 3 tipos de comillas.

1. Comillas dobles: "Hola".
2. Comillas simples: 'Hola'.
3. Backticks (comillas invertidas): `Hola`.

Las comillas dobles y simples son comillas "sencillas" (es decir, funcionan igual). No hay diferencia entre ellas en JavaScript.

Los backticks son comillas de "funcionalidad extendida". Nos permiten incrustar variables y expresiones en una cadena de caracteres encerrándolas en `${...}`

```
let name = "John";
// incrustar una variable
console.log( `Hola, ${name}!` );
console.log("Hola, "+name+"!");
```

3. Variables y datos

3.6. Alternativas a console.log

```
console.log("%ciHola, mundo!", "color: blue; font-size:

//Truco 2. Console.info, console.warn, console.error
console.info("Esto es un mensaje informativo.");
console.warn("Esto es una advertencia.");
console.error("Esto es un mensaje de error.");

// Truco 3. Console.table
const personas = [
  { nombre: "Juan", edad: 30, ciudad: "Madrid" },
  { nombre: "Ana", edad: 25, ciudad: "Barcelona" },
  { nombre: "Pedro", edad: 35, ciudad: "Valencia" }
];
console.log(personas)
console.table(personas);
```

Usa **%c** y agrega las propiedades CSS

4. Operadores y control de flujo

4.1. Operadores

Aritméticos: + - * / % **

Comparación: > < >= <= == === != !==

Lógicos: && || !

4. Operadores y control de flujo

4.2. == frente a ===

==: Compara permitiendo conversiones de tipo.

```
5 == "5"   //resultado: true
```

===: Compara el valor y el tipo.

```
5 === "5"  //resultado: false
```

En nuestros programas utilizaremos preferentemente: === y !==

4. Operadores y control de flujo

4.3. Condicional if / else if / else

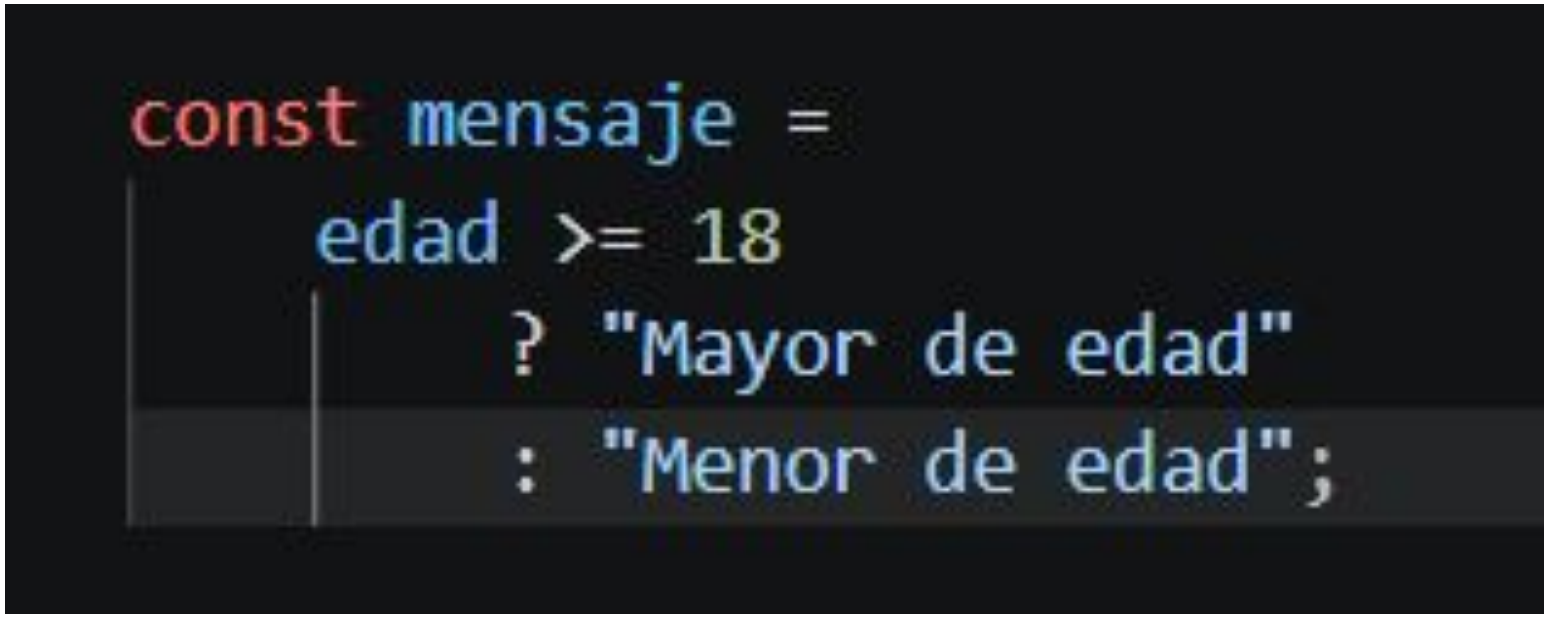
```
const nota = 7;

if (nota >= 9) {
    console.log("Sobresaliente");
} else if (nota >= 7) {
    console.log("Notable");
} else if (nota >= 5) {
    console.log("Aprobado");
} else {
    console.log("Suspenso");
}
```

4. Operadores y control de flujo

4.4. Operador ternario

condicion ? valorSiTrue : valorSiFalse



4. Operadores y control de flujo

4.5. Switch

```
const opcion = 2;

switch (opcion) {

    case 1:
        console.log("Alta");
        break;

    case 2:
        console.log("Modificar");
        break;

    case 3:
        console.log("Eliminar");
        break;

    default:
        console.log("Opción incorrecta");
}
```

4. Operadores y control de flujo

4.6. Bucles

for

while

for…of

```
const nombres = ["Ana", "Luis", "Marta"];

for (const nombre of nombres) {
    console.log(nombre);
}
```

5. Funciones

5.1. Funciones anónimas

Las funciones **anónimas o lambda** son aquellas que no han sido declaradas con un nombre y se alojan en el interior de una variable. Previenen el hoisting.

```
/*FUNCIÓN ANÓNIMA*/
let result3 = function (name, year = 2025){
    let copyright = name + " - " + year;
    return copyright;
}

console.log(result3("DAW"));
```

5. Funciones

5.2. Callbacks

Una función puede recibir otra función como parámetro. A esta función se le denomina **callback**.
Las funciones **anónimas** se utilizan habitualmente como callbacks.

```
/*FUNCIÓN CALLBACK*/
let result2 = function (name, year, formatter){
    let copyright = formatter(name, year);
    return copyright;
}
let formatter = function(name, year){
    return name + " | " + year;
}
console.log(result2("DAM", 2025, formatter));
```

5. Funciones

5.3. Funciones auto-invocadas

Las funciones **auto-invocadas** son funciones anónimas que se invocan automáticamente en el mismo momento en que se ejecuta el script.

```
/*FUNCIÓN AUTOINVOCADA*/
(function(name,year){
    console.log(name, year);

})("Pilar", 2028);
```

5. Funciones

5.4. Funciones Arrow =>

- Las Arrow functions, funciones flecha o «fat arrow» son una forma corta de escribir funciones (a partir de ECMAScript 6). Se elimina la palabra function y se añade => antes de abrir las llaves.
- Con este nuevo enfoque, estamos creando una función en el interior de una variable, lo que nos permitirá posteriormente ejecutar la variable (como si fuera una función).

```
function suma(a, b){
    return a+b;
}

//MISMA FUNCIÓN EXPRESADA CON =>
const suma2 = (a, b) => {
    return a+b;
}

//EN UNA SOLA LÍNEA CUANDO SÓLO HAY UN RETURN
const suma3 = (a, b) => a+b;
```

function (){ }

()=> {}

Más info: https://javascript.info/arrow-functions-basics

5. Funciones

5.4. Funciones Arrow =>

Ventajas de usar =>

- Si el cuerpo de la función sólo tiene una línea, podemos omitir las llaves ({}).
- Además, en ese caso, automáticamente se hace un return de esa única línea, por lo que podemos omitir también el return.
- En el caso de que la función no tenga parámetros, se indica como : () =>.
- En el caso de que la función tenga un solo parámetro, se puede indicar simplemente el nombre del mismo: e =>.
- En el caso de que la función tenga 2 ó más parámetros, se indican entre paréntesis: (a, b) =>.
- Si queremos devolver un objeto, que coincide con la sintaxis de las llaves, se puede englobar con paréntesis: ({name: 'Manz'}).

Ejemplos de =>

```
//CON UN SOLO PARAM SE OMITEN LOS ( )
const cuadrado = x => x*x;
console.log(cuadrado(5));
```

```
//SIN PARAMS SE USA ( )
const saludo = ( ) => "Hola, mundo";
console.log(saludo);
```

```
//USANDO MAP
const numeros = [1, 2, 3, 4];
const dobles = numeros.map(n => n*2);
console.table(dobles);
```

```
//USANDO FILTER
const edades = [12, 16, 18, 34, 29, 10];
const mayoresEdad = edades.filter(edad => edad>= 18);
console.table(mayoresEdad);
```

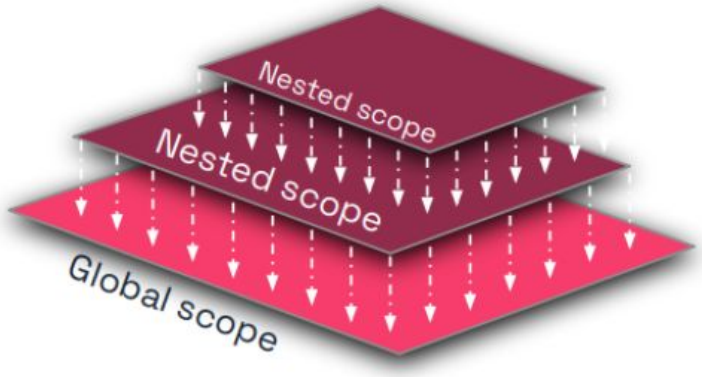
EJERCICIOS: Transformar a =>

```
function sumar(a, b) {
    return a + b;
}

function cuadrado(numero) {
    return numero * numero;
}

function saludar(nombre) {
    return `Hola ${nombre}`;
}
```

6. Ámbitos (scope) en JS



Distinguimos entre dos tipos de ámbitos:

- Global (global scope): disponible en todo el script.
- Local (local scope): dentro de éste, encontramos ámbito de función y ámbito de bloque.

El ámbito determina el **alcance de las expresiones y valores dentro de un contexto** y tiene una jerarquía unidireccional, en la que **el ámbito global es el padre y los demás son los hijos.**

Los hijos pueden acceder a la información de los padres, pero no al revés.

```
/*ÁMBITO GLOBAL*/
let name="Pilar";

function printName(){
    console.log(name); //tacha la variable name y pone que es deprecated pero lo ejecuta porque el hijo puede acceder a la variable global
}
printName();


/*ÁMBITO LOCAL DE FUNCIÓN*/
function printLastName(){
    let lastname="gonzalez";
}
printLastName();
console.log(lastname);  //error lastname no está definido porque la variable lastname es de ámbito local a la función
```

```
/*ÁMBITO LOCAL DE BLOQUE*/
function printAddress(){
    let lastname="gonzalez";
    if (lastname=="gonzalez"){
        let address="Velazquez";
    }
    console.log(address);  //error address no está definido. Si defino la variable en el padre sí podría acceder
}
printAddress();

if (true){
    let text="Hola y adiós";
}
console.log(text);
```

Ejemplos para practicar los ámbitos

7. Arrays

Los arrays en JavaScript tienen estas peculiaridades:

- **No** es necesario declarar el **tipo** de los elementos.
- Pueden contener **distintos tipos** de datos.
- Su **tamaño** puede cambiar **dinámicamente**.
- Pueden **almacenar funciones, objetos y otros arrays**.

```
const datos = [25, "Sevilla", true, [1, 2, 3]];
```

```
let nombres = [];
```

7. Arrays

7.1. Métodos básicos: push(), pop(), unshift(), shift()

```
const frutas = ["manzana", "pera", "plátano"];

frutas.push("naranja");      // Añade al final
frutas.pop();                // Elimina el último
frutas.unshift("fresa");     // Añade al principio
frutas.shift();              // Elimina el primero

console.table(frutas);
console.log(frutas.length);
console.log(frutas.includes("pera2"));
console.log(frutas.indexOf("pera"));
```

7. Arrays

7.2. foreach()

foreach() ejecuta una función por cada elemento, pero **no devuelve un nuevo array.**

```
const numeros = [1, 2, 3, 4, 5];

numeros.forEach(numero => {
    console.log(numero);
});
```

7. Arrays

7.3. includes(), some()

```
const personajes = [
    "Tokio",
    "Berlín",
    "Nairobi",
    "Denver"
];

console.log(personajes.includes("Nairobi"));
```

7. Arrays

7.4. find()

find() devuelve el primer objeto que cumple la condición. Si no hay coincidencia, devuelve **undefined**.

```
const usuarios = [
    { id: 1, nombre: "Ana" },
    { id: 2, nombre: "Luis" },
    { id: 3, nombre: "Marta" }
];

const usuario = usuarios.find(usuario => usuario.id === 2);

console.log(usuario);
```

7. Arrays

7.5. map(), filter()

map() permite recorrer un array y crear otro array con los resultados de transformar cada elemento.

filter() crea un nuevo array con los elementos que cumplen una condición.

```
const edades = [15, 16, 17, 20, 22];

const mayores = edades.filter(
    edad => edad >= 18
);

console.log(mayores);
```

```
users.map(user => {
  console.log(`${user.username} - ${user.email}`);
});
```

7. Arrays

7.5. map(), filter()

```
const alumnos = [
    { nombre: "Ana", nota: 8 },
    { nombre: "Luis", nota: 4 },
    { nombre: "Marta", nota: 10 },
    { nombre: "Carlos", nota: 3 }
];
const aprobados = alumnos.filter(alum=>alum.nota>=5)
    .map(alum=>alum.nombre);
;
console.table(aprobados);
```

Ejemplo combinado con map y filter

7. Arrays

7.6. reduce()

reduce() recorre un array y permite obtener un **único resultado** a partir de sus elementos.

Sintaxis:

```
array.reduce((acumulador, elemento) => {
        return resultado;
}, valorInicial);
```

***valorInicial* es opcional,** si se omite, el acumulador toma el primer elemento del array y la iteración comienza en el segundo

```
const numeros3 = [2, 3, 4];

const resultado = numeros3.reduce(
    (acumulador, numero) => acumulador * numero,
    1
);

console.log(resultado); // 24
```

7. Arrays

7.6. reduce()

```
const productos = [
    { nombre: "Teclado", precio: 30, stock: 5 },
    { nombre: "Ratón", precio: 20, stock: 10 },
    { nombre: "Monitor", precio: 150, stock: 2 }
];

const valorStock = productos.reduce((total, producto) => {
    return total + producto.precio * producto.stock;
}, 0);

console.log(valorStock);
```

7. Arrays

7.6. reduce()

Ejercicio:

```
const videojuegos = [
    { nombre: "Zelda", precio: 60 },
    { nombre: "Minecraft", precio: 30 },
    { nombre: "FIFA", precio: 50 },
    { nombre: "Hades", precio: 25 }
];
```

Calcula:

1. El precio total.
2. El precio medio.

7. Arrays

7.7. Operador spread ...

El **operador *Spread*** (...) permite expandir los elementos de un array o las propiedades de un objeto.

Se utiliza para **crear nuevos arrays u objetos** a partir de otros.

```
const numeros = [1, 2, 3];
const nuevosNumeros = [...numeros, 4, 5];

console.log(numeros);  // [1, 2, 3]
console.log(nuevosNumeros);  // [1, 2, 3, 4, 5]
```

```
const persona = { name: 'Juan', age: 25 };
const personaActualizada = { ...persona, age: 26 };

console.log(persona); // { name: 'Juan', age: 25 }
console.log(personaActualizada); // { name: 'Juan', age: 26 }
```

7. Arrays

7.7. Operador spread ...

Combinar arrays:

```
const nombres1 = ["Ana", "Luis"];
const nombres2 = ["Marta", "Carlos"];

const todos = [...nombres1, ...nombres2];

console.log(todos);  // ["Ana", "Luis", "Marta", "Carlos"]
```

7. Arrays

7.8. Operador Rest ...

El **operador *Rest* (...)** permite **recoger** los elementos restantes y agruparlos en una variable.

Se puede utilizar en **parámetros** de funciones y en **desestructuración** de arrays y objetos.

```
const persona = {
  nombre: 'Luis',
  edad: 30,
  ciudad: 'Bogotá',
  pais: 'Colombia'
};

const { nombre, ...detalles } = persona;

console.log(nombre);   // 'Luis'
console.log(detalles); // { edad: 30, ciudad: 'Bogotá', pais: 'Colombia' }
```

7. Arrays

7.8. Operador Rest ...

```
function crearUsuario(nombre, email, ...roles) {
    return {
        nombre: nombre,
        email: email,
        roles: roles
    };
}

const usuario2 = crearUsuario(
    "Ana",
    "ana@email.com",
    "usuario",
    "editor",
    "administrador"
);

console.log(usuario2);
```

7. Arrays

7.8. Operador Rest ...

Ejercicio:

Crea una función ***crearVideojuego()*** que reciba:

el nombre del videojuego

el precio

uno o varios géneros

Utiliza **Rest** para recoger todos los géneros.

7. Arrays

7.7. join(), split()

join() es un método que une todos los elementos de un array en una sola cadena de texto. Se puede especificar un separador (como una coma o un guión) para unirlos. Si no, usa coma por defecto.

split() convierte una string en array

```
const texto = "manzana,banana,naranja";
const array = texto.split(',');  // String a array: ['manzana', 'banana', 'naranja']
const nuevoTexto = array.join('-');  // Array a string: "manzana-banana-naranja"
console.log(nuevoTexto);
```

8. Entrada de datos en node: readline

JS codeKata1.js > ...

```
1   const readline = require("readline");
2
3   const rl = readline.createInterface({
4       input: process.stdin,
5       output: process.stdout
6   });
7
8   rl.question("¿Cómo te llamas? ", nombre => {
9
10      console.log(`Hola ${nombre}`);
11
12      rl.close();
13  });
```

Problems Output Debug Console Terminal Ports

```
PS C:\JS> node .\codeKata1.js
¿Cómo te llamas? pilar
Hola pilar
PS C:\JS>
```

En *Node.js* para leer datos desde la consola utilizamos el módulo readline.

8. Entrada de datos en HTML: prompt()

mesas.js ● imprime.html ● JS berghain.js 1

prime.html > html

```html
<!DOCTYPE html>
<html lang="en">
  <head>
    <title>Imprime un usuario</title>
  </head>
  <body>

  <script src="./berghain.js"></script>
  </body>
</html>
```

berghain.js > ...

```
1   const usuario={
2       edad:1,
3       conoceDJ:true,
4       ropaNegra:true
5   }
6
7   function getData() {
8     usuario.edad = Number(prompt("¿Qué edad tienes?"));
9     usuario.conoceDJ=prompt("¿Conoces al DJ?");
10    usuario.ropaNegra=prompt("¿Color de ropa?");
11  }
12  getData();
13  alert(usuario.conoceDJ);
```



RETO: LA CASA DE PAPEL

¿Has visto la serie española La casa de papel?

Los personajes tienen nombres de capitales:

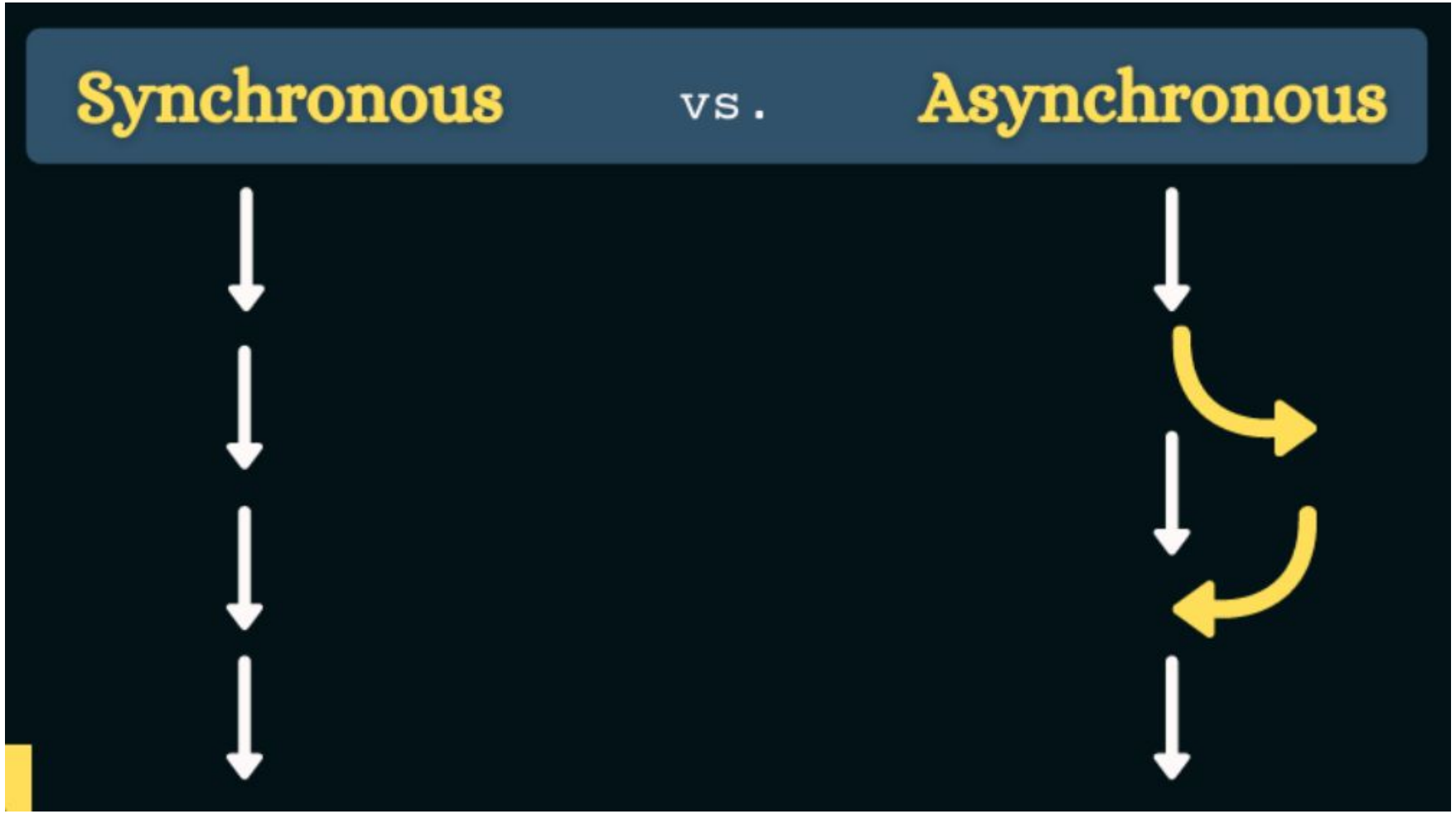
Nairobi, Berlín, Helsinki, Tokio, Río, Denver, Lisboa, Moscú, Estocolmo.

Crea un código JavaScript en el que **se solicite un nombre de capital** al usuario y se le indique **si es un personaje de la serie o no**.



- Hazlo con un **switch** case y con **arrays**.
- Utiliza ***prompt*** (embebiendo en html) o ***readline*** (nativa de ***Node***, usar ***node tu_archivo.js*** para ejecutarlo en el terminal).
- **Normaliza** las entradas: espacios en blanco, tildes, mayúsculas y minúsculas.

9. Sincronía

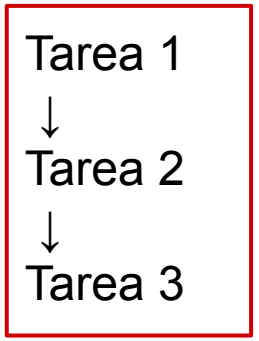


9. Sincronía

9.1. Sincronía vs. asincronía

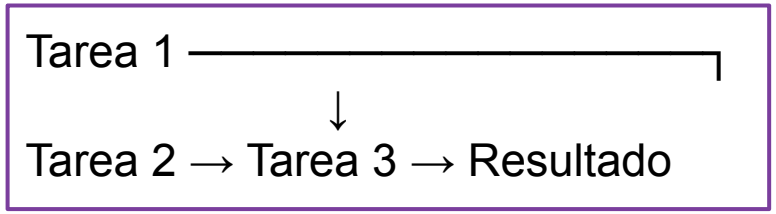
Sincronía:

- En un modelo síncrono, las operaciones se ejecutan de manera **secuencial**. El programa espera a que una operación se complete antes de pasar a la siguiente.
- Mientras una operación síncrona está en curso (lectura de archivo, solicitud a una BD), el **hilo principal de la aplicación se bloquea**, impidiendo que otras tareas se ejecuten.



Asincronía:

- En un modelo asíncrono, **las operaciones no bloquean el hilo principal**.
- El programa puede seguir ejecutando otras tareas mientras **la operación asíncrona se resuelve en segundo plano.**



9. Asincronía en JavaScript

9.2. Asincronía en JavaScript

JavaScript es un lenguaje de **un solo hilo**, lo que significa que sólo puede ejecutar una tarea a la vez en su hilo principal. Sin embargo **puede manejar operaciones asíncronas** de forma eficiente, ejecutándose en segundo plano. El siguiente código simula una operación asíncrona con **setTimeout**:

```
console.log("Inicio");
setTimeout(() => {
  console.log("Operación asíncrona completada");
}, 2000);
console.log("Fin");
```

10. Promises

10.1. Promises. Estados

Una Promise representa el **resultado futuro de una operación asíncrona**. Ejemplo obtener datos de una API o base de datos.

Admiten tres estados posibles:

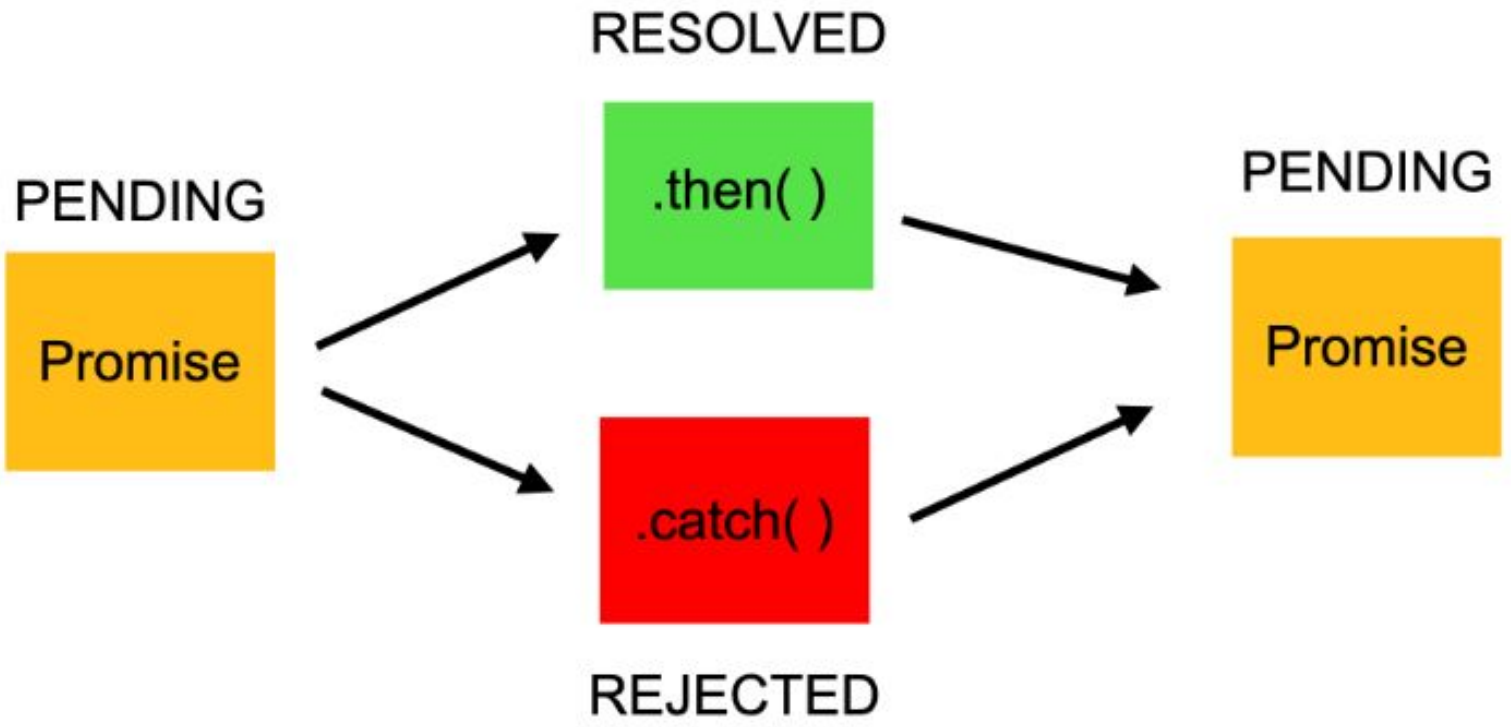
- **Pending**: estado inicial, antes de que la promesa tenga éxito o falle.
- **Fulfilled**: Promesa completada.
- **Rejected**: promesa fallida.

Métodos: **.then(), .catch(), finally.**

Más info: https://es.javascript.info/promise-basics

10. Promises

10.1. Promises. Estados



10. Promises

10.2. then

Se crea con el constructor ***Promise***, que toma una función con dos parámetros: ***resolve*** (para cumplir la promesa) y ***reject*** (para rechazarla).

.then se ejecuta si la promise se cumple, ***.catch*** gestiona el error, ***.finally*** se ejecuta independientemente del resultado.

```
const promesa = new Promise((resolve, reject) => {
  setTimeout(() => resolve("Datos cargados!"), 1000);
  // setTimeout(() => reject("Error al cargar"), 1000);
});
promesa
  .then(data => console.log(data))
  .catch(error => console.error(error));
```

10. Promises

10.3. async / await

La sintaxis **async/await** es una forma más **legible** de trabajar con promesas (***syntactic sugar***).

Una función declarada con **async** siempre **devuelve una promesa**, y **await espera a que una promesa se resuelva.**

Para el manejo de errores se utiliza **try/catch**.

```
const fetchUsuarios = async () => {
    const datos = await new Promise(resolve =>
        setTimeout(() => resolve(["Ana", "Luis"]), 1000)
    );
    return datos;
};
fetchUsuarios().then(usuarios => console.log(usuarios));
```

```
function getNumber(){
    return new Promise((resolve, reject) => {
        setTimeout(()=> {
            try{
                resolve(10)
            }catch(error){
                reject(error);
            }
        }, 1000);
    });
}
async function useNumber(){
    try{
        let number = await getNumber();
        console.log("Número recibido", number);
    }catch (error){
        console.error("Error: ", error);
    }
}

useNumber();
```

RETO: En este código *getNumber* siempre resuelve con éxito *(resolve(10)).* Modifícalo para que utilice ***Math.random()*** y utiliza un condicional para que la promesa se resuelva como *fulfilled* o como *rejected*. Que sea fulfilled si el número es > 0,5

RETO: ¿PUEDES ENTRAR EN EL CLUB BERGHAIN?

Crea una aplicación que simule el acceso al famoso Berghain.



- Debe pedir al usuario su edad, si conoce al DJ de la noche y si encaja con el ambiente. Los datos se almacenarán en un **objeto usuario**.
- Para poder entrar debe cumplir las tres condiciones.
- La función ***comprobarEntrada(usuario)*** deberá devolver una Promise que gestione si puede o no entrar.
- Si no puede entrar, se deberán guardar todos los motivos del rechazo en un array y mostrarlos al usuario.
- Utiliza readline, normalización de strings, objetos, arrays, funciones, Promise, resolve(), reject(), .then() y .catch().

11. JSON: JavaScript Object Notation

- **JSON** es un **formato de texto para intercambiar datos entre aplicaciones.**
- Se utiliza habitualmente para comunicar un cliente con un servidor mediante una API.
- Su estructura se parece a la de los objetos y arrays de JavaScript.
- **JSON no es un objeto JavaScript: es un formato de datos.**
- Tiene dos métodos estáticos:
  - **parse**: transforma un texto JSON a un objeto JSON dentro de JavaScript.
  - **stringify**: toma un objeto JavaScript como argumento y lo transforma en un texto JSON.

11. JSON: JavaScript Object Notation

```
{
    "text": "siempre entre comillas",
    "number": 10,
    "decimal": 10.23,
    "array": [
        "valor",
        "otro valor"
    ],
    "object": {
        "property": "texto",
        "boolean": true
    },
    "date": "2023-06-01",
    "nullValue": null,
    "hierarchical": [
        { "with": "object" },
        false,
        [ "more", 1, "values", null, { "with": "object" } ]
    ]
}
```

Sintaxis de JSON

```
{
  "propiedad": valor,
  "propiedad": valor,
  "propiedad": valor
}
```

- Claves y textos: comillas dobles.
- Números y booleanos: sin comillas.
- Clave y valor: separados por :
- Propiedades: separadas por comas.

11. JSON: JavaScript Object Notation

```js
//objeto JS a JSON
const usuario = {
    nombre: "Ana",
    edad: 22,
    ciudad: "Sevilla"
};

const usuarioJSON = JSON.stringify(usuario);

console.log(usuarioJSON);
console.log(typeof(usuarioJSON));
```

```js
//JSON a objeto JS
const datosJSON = `
{
    "nombre": "Carlos",
    "edad": 25,
    "curso": "2DAM"
}
`;

const usuario2 = JSON.parse(datosJSON);

console.log(usuario2.nombre);
console.log(usuario2.edad);
console.log(usuario2.curso);

//Por qué es necesario hacer un parse()... al imprimir sale undefined
console.log(datosJSON.nombre);
```

stringify:
objeto JavaScript → texto JSON

parse:
texto JSON → objeto JavaScript

11. JSON: JavaScript Object Notation

```
//EJ3 con array
const productos = [
    {
        nombre: "Teclado",
        precio: 30
    },
    {
        nombre: "Monitor",
        precio: 180
    },
    {
        nombre: "Ratón",
        precio: 20
    }
];
```

Ejemplo propuesto:

- **Convierte productos en JSON y muéstralo.**
- **Convierte de nuevo el JSON a un objeto JavaScript.**
- **Utiliza filter() para obtener los productos de más de 25 €.**
- **Muestra los productos resultantes.**

12. Lectura de JSON local con Node.js

alumnos.json

```json
[
    { "nombre": "Ana", "nota": 8 },
    { "nombre": "Luis", "nota": 4 },
    { "nombre": "Marta", "nota": 9 },
    { "nombre": "Pedro", "nota": 3 },
    { "nombre": "Lucía", "nota": 6 }
]
```

main.js

```js
const fs = require("fs");

// Leer el archivo
const contenido = fs.readFileSync("alumnos.json", "utf8");

console.log(contenido);
console.log(JSON.parse(contenido));

const alumnos = JSON.parse(contenido);

console.log(alumnos[2].nota);
```

Ejecución: ***node main.js***

EJERCICIO: Buscador ofertas de empleo

Utilizando el archivo ***ofertas.json***, desarrolla un programa en ***Node.js*** que permita consultar ofertas por sector profesional. Solicita el sector mediante **readline**.

a) Define ***filtrarPorSector(ofertas, sector)*** para obtener las ofertas del sector indicado, sin distinguir entre mayúsculas y minúsculas.
b) Muestra la empresa y el salario de las ofertas encontradas. Si no hay resultados, informa de ello.
c) Obtén un nuevo array con las ofertas activas del sector consultado, ordenadas de mayor a menor salario, sin modificar el array original.
d) Comprueba si alguna oferta activa supera los 35.000 € anuales e informa del resultado.

ofertas.json

13. fetch() — Obtener datos de una API

fetch() permite realizar una **petición a un recurso**, normalmente una API, y obtener su respuesta.

fetch() **devuelve una Promise**, usamos await para esperar la respuesta.

Sintaxis:

const response = await fetch(url);

const datos = await response.json();

- **response** → variable que contiene la respuesta del servidor.
- **response.json()** → obtiene los datos JSON de la respuesta.
- **datos** → variable que contiene los datos

```javascript
async function fetchUsers() {
  try {
    const response = await fetch('https://jsonplaceholder.typicode.com/users');
    if (!response.ok) {
      throw new Error(`HTTP error: ${response.status}`);
    }
    const users = await response.json();
    console.table(users, ['id', 'name', 'username', 'email', 'phone']);
  } catch (error) {
    console.error('Error fetching users:', error);
  }
}

fetchUsers();
```

MS OUTPUT DEBUG CONSOLE TERMINAL PORTS Filter (e.g. text,

rogram Files\nodejs\node.exe --experimental-network-inspection .\fetchAPI.js

) [{…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}, {…}]

ndex)	id	name	username	email
	1	'Leanne Graham'	'Bret'	'Sincere@april.biz'
	2	'Ervin Howell'	'Antonette'	'Shanna@melissa.tv'
	3	'Clementine Bauch'	'Samantha'	'Nathan@yesenia.net'
	4	'Patricia Lebsack'	'Karianne'	'Julianne.OConner@kory.org

fetching con async function y await

URL: https://jsonplaceholder.typicode.com/users

```
fetch_then.js > ...
1   function fetchUsers() {
2       fetch("https://jsonplaceholder.typicode.com/users")
3           .then(response => {
4               if (!response.ok) {
5                   throw new Error(`HTTP error: ${response.status}`);
6               }
7
8               return response.json();
9           })
10          .then(users => {
11              console.table(users, [
12                  "id",
13                  "name",
14                  "username",
15                  "email",
16                  "phone"
17              ]);
18          })
19          .catch(error => {
20              console.error("Error fetching users:", error);
21          });
22  }
23
24  fetchUsers();
```

fetching con then

URL: https://jsonplaceholder.typicode.com/users

Primer **.then()**: comprueba la respuesta HTTP. **response.json()** convierte los datos y los pasa al siguiente .then().
Segundo **.then()**: procesa los usuarios. **.catch():** gestiona los errores.

filter_city.js > ...

```
function fetchUsers() {
    fetch("https://jsonplaceholder.typicode.com/users")
        .then(response => {
            if (!response.ok) {
                throw new Error(`HTTP error: ${response.status}`);
            }

            return response.json();
        })
        .then(users => {

            const usuariosFiltrados = users.filter(
                user => user.address.city === "South Elvis"
            );

            usuariosFiltrados.forEach(user => {
                console.log("Nombre:", user.name);
                console.log("Email:", user.email);
                console.log("Ciudad:", user.address.city);
            });
        })
        .catch(error => {
            console.error("Error fetching users:", error);
        });
}

fetchUsers();
```

14. Módulos: import / export

En JavaScript, un **módulo** es un archivo que contiene código reutilizable y que puede ser importado o exportado a otros archivos. Esto permite mantener una aplicación organizada, escalable y fácil de mantener.

export: permite hacer accesible una función o variable desde otro archivo.

import: permite utilizar lo que otro archivo ha exportado.

```
// util.js
export const saludar = nombre => `Hola, ${nombre}!`;
// main.js
import { saludar } from './util.js';
console.log(saludar("Ana"));
```

14. Módulos: import / export

Existen dos formas de exportar módulos: con nombre y por defecto (export default)

Exportación con nombre	Exportación por defecto
export const saludar = ...	export default function Saludo() { ... }
import { saludar } from './util.js'	import Saludo from './Saludo.js'
Se importa usando el mismo nombre. Un archivo puede tener varias.	Se puede elegir el nombre al importar. Un archivo solo puede tener una exportación por defecto.

RETO: Crea los siguientes archivos *util.js* y *modulos.js*

```js
// util.js
export const obtenerProductos = () => [
  { id: 1, nombre: "Laptop", precio: 1000 },
  { id: 2, nombre: "Teléfono", precio: 500 }
];

export const calcularTotal = productos => productos.reduce((sum, { precio }) => sum + precio, 0);
```

Exporta una función de ***util.js*** que obtenga los productos más caros que 600 y úsala en ***modulos.js***
Para usar import/export en Node.js necesitas ejecutar ***npm init -y***. Se genera ***package.json***, agrega la línea *"type": "module"*

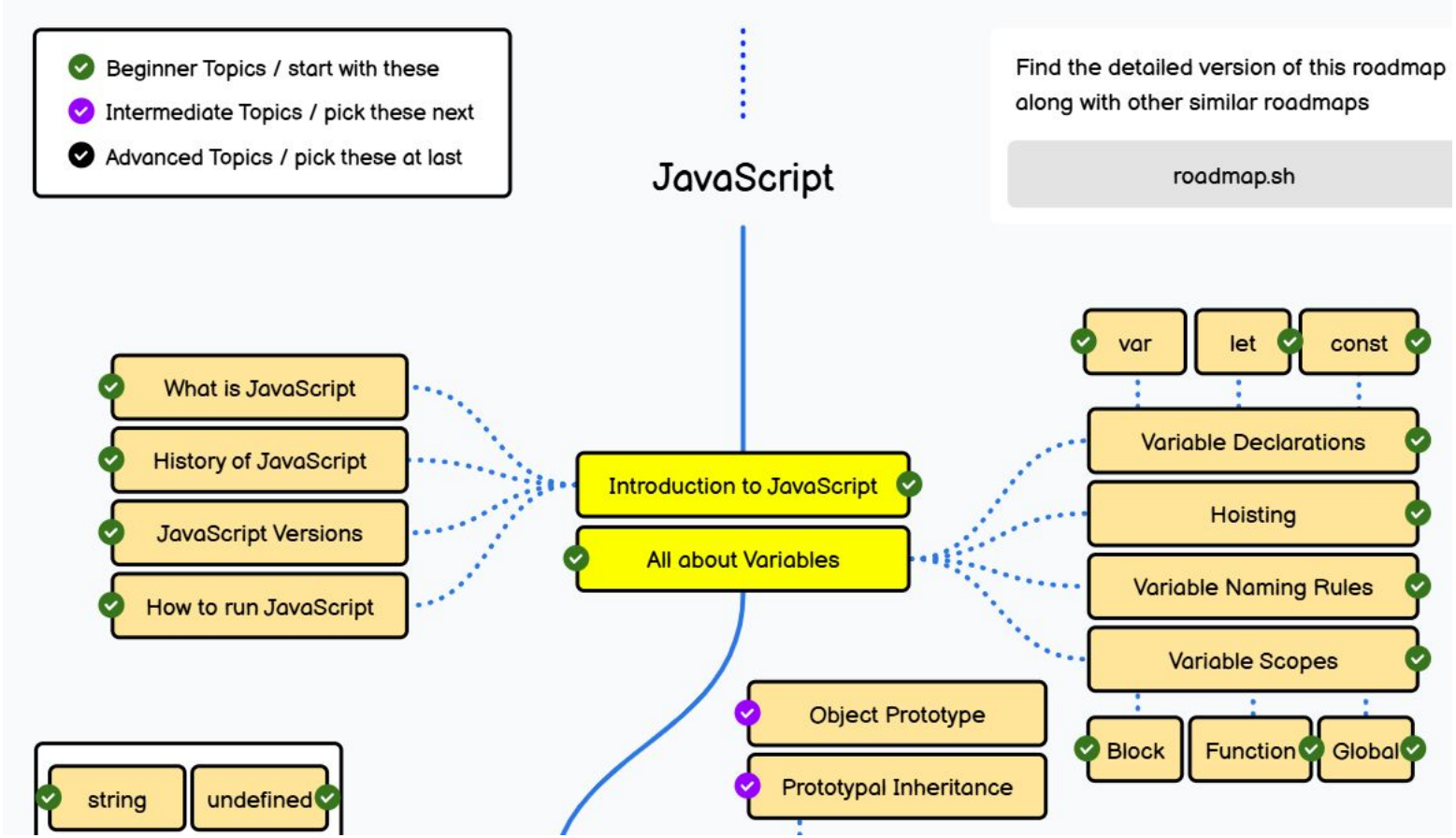
```js
// modulos.js
import { obtenerProductos, calcularTotal } from './util.js';

const ComponenteCarrito = () => {
  const productos = obtenerProductos();
  const total = calcularTotal(productos);
  console.log("Productos:", productos);
  console.log("Total:", total);
};
ComponenteCarrito();
```

Alternativa: ***npm.cmd init -y***

Fin.-.- Por ahora

https://roadmap.sh/javascript



ANEXOS: Métodos y propiedades de String, Arrays, Date

String

Propiedad	Significado	Ejemplo
length	Tamaño del texto	longitud=texto.length;
Métodos	**Significado**	**Ejemplo**
charAt(*posición*)	**Devuelve** el carácter situado en la posición.	letra=texto.CharAt(2);
indexOf(texto[,inicio])	**Devuelve** dónde comienza el texto dentro de la cadena. Podemos comenzar a buscar a partir de inicio. Si no lo encuentra devuelve -1	pos=texto.indexOf("cama");
replace(texto1,texto2)	**Devuelve** una cadena en la que busca texto1 y lo reemplaza por texto2	texto.replace("pro","por");
split([Separa [,límite])	**Devuelve** el texto en un array de textos usando el texto Separa para distinguir los elementos. Limite limita el número de substrings que creamos	nuevoTexto=texto.split(" ");
substr(inicio,final)	**Devuelve** la subcadena comenzando en la posición *inicio* hasta la posición *final*	cade=texto.substr(3,5);
trim()	**Devuelve** una nueva cadena en la que se elimina los espacios en blanco del inicio y del final de la cadena	TextoNuevo=texto.trim();
toLowerCase()	Devuelve una cadena en la que se convierte a minúsculas todas las letras del texto	minus=texo.toLowerCase();
toUpperCase()	Devuelve una cadena en la que se convierte a mayúsculas todas las letras del texto	mayus=texto.toUpperCase();

Arrays

Propiedad	Significado	Ejemplo
length	Tamaño del array	indice=tabla.length;
Métodos	**Significado**	**Ejemplo**
concat(array1,array2,..)	Devuelve la union de los arrays	dato=tabla.concat(tabla1,tabla2)
indexOf(elemento)	Devuelve la posición donde se encuentra el elemento. Devuelve -1 si no lo encuentra	i=tabla.indexOf(34);
join()	Devuelve el contenido de un array en forma de cadena de texto. Los elementos estarán separados por , o por el texto que se pase como parámetro	cadena=tabla.join("+");
lastIndexOf(elemento)	Devuelve la posición comenzando por el final donde se encuentra el elemento. Devuelve -1 si no lo encuentra	var a = lastIndexOf("x");
pop()	Elimina el último elemento del array y devuelve el elemento	dato=tabla.pop();
shift()	Elimina el primer elemento del array y devuelve el elemento	dato=tabla.shif();
push(elemento)	Inserta un elemento por el final del array y devuelve el nuevo tamaño	i=tabla.push(dato);
reverse()	Invierte el orden de los elementos del array	Tabla.reverse()
sort()*	Ordena los elementos del array	Elementos.sort()

Date

Método	Significado
getDate()	Devuelve el número del día del mes (1-31)
getDay()	Devuelve el número del día de la semana (0-6)
getFullYear()	Devuelve el año (yyyy)
getHours()	Devuelve la hora (0-23)
getMilliseconds()	Devuelve los milisegundos (0-999)
getMinutes()	Devuelve los minutos (0-59)
getMonth()	Devuelve el número de mes (0-11)
getSeconds()	Devuelve los segundos (0-59)
getTime()	Devuelve los millisegundos desde el 1 de enero de 1970